SQL Procedure Exercises

1. Test the procedure with different inputs

Run these calls separately and record whether each succeeds, whether a payment row is added, and the resulting balance for student `1`.

```
CALL record_payment(1, 49.00);
CALL record_payment(1, -25.00);
```

For the second call, identify whether the rejection comes from code inside `record_payment` or from a rule on the `payments` table.

2. Return a grade with an `OUT` parameter — 15 minutes

Run each `CALL` and inspect `@student_grade` immediately afterward.

```
CALL get_student_grade(1, 101, 'Fall 2026', @student_grade);
SELECT @student_grade AS grade;

CALL get_student_grade(2, 101, 'Fall 2026', @student_grade);
SELECT @student_grade AS grade;

CALL get_student_grade(1, 101, 'Spring 2027', @student_grade);
SELECT @student_grade AS grade;
```

For each call, identify whether an enrollment exists and explain the output.

3. Build a grade report and debug its filters — 10 minutes

Call `get_student_grade` for student `1`, course `101`, and `Fall 2026`. Then write a query that displays the **student ID, student name, course name, semester, and `@student_grade`**.

Use an implicit join. List `enrollments`, `student_accounts`, and `courses` in `FROM`, put the two join conditions in `WHERE`, and add filters for the three inputs passed to the procedure.

Answers:

1. Test the procedure with different inputs

Assuming student 1 starts with the previous exercise's balance of **649.00**, and a payment trigger subtracts accepted payments:

Call	Succeeds?	Payment row added?	Resulting balance
`CALL record_payment(1, 49.00);`	Yes, assuming no other validation fails	Yes	600.00
`CALL record_payment(1, -25.00);`	No, assuming negative payments are prohibited	No	649.00

- The procedure attempts the insert and a constraint such as `CHECK (amount > 0)` rejects it, the rejection comes from the `payments` table.

2. Return a grade with an OUT parameter

The exact grades and whether each enrollment exists require the enrollment data.

Call inputs	Expected output
Student 1, course 101, Fall 2026	A
Student 2, course 101, Fall 2026	NULL
Student 1, course 101, Spring 2027	NULL

Assuming the procedure uses `SELECT grade INTO` its `OUT` parameter, a missing enrollment normally leaves the output **`NULL`**, unless the procedure defines different handling. An existing enrollment can also have a `NULL` grade if no grade has been assigned.

3. Build a grade report using an implicit join

```
CALL get_student_grade(
    1, 101, 'Fall 2026', @student_grade
);

SELECT
    s.student_id,
    s.student_name,
    c.course_name,
    e.semester,
    @student_grade AS grade
FROM enrollments AS e
JOIN student_accounts AS s
    ON e.student_id = s.student_id
JOIN courses AS c
    ON e.course_id = c.course_id
WHERE e.student_id = 1
  AND e.course_id = 101
  AND e.semester = 'Fall 2026';
```